

第025讲 区间估计与Bootstrap实验

1. 本次要交付什么

完成实验后，你应能提交一份短报告，包含一个精确Gaussian覆盖证明、两个可完全枚举的例子、一组配对对照和一组分组覆盖对照。每个结果都要写清楚估计目标、独立单位、区间算法和误差来源。不要只交截图或“程序没有报错”。

全部教学数据合成，离线CPU运行。先修为第21至24讲。lecture.pdf解释原理；experiment.ipynb按下面的顺序执行并保留结果；experiment.py提供相同的核心实现；answers.pdf给完整推导与18题解答。默认工作无需网络、密钥或付费API。

推荐先用纸笔完成第3、4步，再运行Notebook核验。预计阅读与实验共90至150分钟，机器计算只是其中很小一部分。当前验收环境实际使用Python 3.12、NumPy 2.3.5、Matplotlib 3.10.8、nbformat 5.11.1、ipykernel 7.4.0。环境文件供安装复现；Anaconda安装、浏览器界面及socket内核传输未在作者环境实测。

2. 准备目录并运行基线

保持本单元整个目录结构。数据路由脚本自己的位置确定，不依赖启动时的工作目录。在终端进入025目录后执行：

```
python experiment.py --self-test
python experiment.py --output-dir outputs
jupyter lab experiment.ipynb
```

在Jupyter里选择Python 3，重新启动内核，然后从第一格顺序运行全部单元格。不要跳过输入检查格，也不要仅看文件中已经保留的旧输出。Notebook不需要预先存在outputs/report.json，会重新运行模拟并把图生成在内存中。脚本默认输出report.json，保留完整的外层区间数组，方便独立重算覆盖率。

若习惯Anaconda，可在能访问conda-forge的环境自行执行：

```
conda env create -f environment.yml
conda activate ml-course-025
python experiment.py --self-test
```

该安装路线没有在作者容器实际执行。不要把路径错误改成删除校验；先确认experiment.py、data目录、Notebook确实在同一级。环境没有预装依赖时按README安装，核心统计函数只需NumPy；图与Notebook另需Matplotlib和Jupyter相关包。

3. 纸笔构造已知标准差区间

观察为(9,10,11,10)，已知 $\sigma=2$ ，假设四次观察独立且Gaussian。

1. 算总和、样本均值和均值的标准误。注意 σ 是题设总体标准差，不是四个数的样本标准差。
2. 用 $z=1.959963984540054$ ，算左右端点。先保留6位小数，展示最后舍入的步骤。
3. 定义随机样本均值和标准化量 Z ，逐步把 $-z \leq Z \leq z$ 改写为参数位于区间内。
4. 若 n 从4变成16且独立性和 σ 不变，计算区间半宽比。解释为什么复制原数据四遍不能支持同一结论。

Python核验：

```
import experiment as e
ci = e.known_sigma_interval([9,10,11,10], 2)
print(ci)
e.require(abs(ci[0] - 8.040036015459946) < 1e-12, 'left endpoint')
```

通过条件：均值10、标准误1、区间约[8.040036,11.959964]。写出95%描述的是重复抽样区间程序，不能只抄数字。

4. 精确枚举与离散分位数

对样本(0,2)列出全部有序索引(1,1)、(1,2)、(2,1)、(2,2)。每项概率都是1/4。计算均值支持、概率、条件期望、条件方差和逆CDF百分位区间。

```
rows = e.exact_bootstrap([0,2])
for row in rows:
    print(row)
values = [row['mean'] for row in rows]
weights = [row['count'] for row in rows]
print(e.weighted_quantile(values, weights, .025))
print(e.weighted_quantile(values, weights, .975))
```

通过条件：支持为0、1、2，计数为1、2、1；方差为1/2；区间为[0,2]。实验代码使用Fraction对实际输入float的值做精确有理数枚举。整数0、2没有二进制表示误差；对0.1，Fraction表示的是该float的精确值，并非十进制1/10。

增加B只能更准确地模拟同一个条件分布。分别用B=99、399、1999和不同种子估计均值与标准差，保留和精确值的误差。不要据某一次误差较小便声称该种子更好。改成三条样本(0,1,4)，先数清 $3^3 = 27$ 个等概率结果，再检查枚举总计数。

5. 对照置信区间与可信区间

接第24讲：先验Beta(2,2)，八次观察三次成功，后验Beta(5,7)。计算等尾95%可信端点，并代回CDF。

```
qlo = e.beta_quantile_integer(.025, 5, 7)
qhi = e.beta_quantile_integer(.975, 5, 7)
print(qlo, qhi)
print(e.beta_cdf_integer(qlo, 5, 7), e.beta_cdf_integer(qhi, 5, 7))
```

通过条件：端点约0.1674880941与0.6920952850，CDF分别约0.025与0.975。写出带先验和模型条件的完整后验概率解释。说明为什么这个后验质量不能证明每个固定p的95%频率覆盖，尤其注意p=0边界。

整数形状Beta接口只支持a、b各在1至40；分位数q限制在[1e-6,1-1e-6]。不要在本实验里输入任意实数形状并声称同一有限和依然成立。需要一般形状时应选择合适库并另行核验。

6. 不靠模拟发现Wald失败

取n=20、真实p=0.02。计算全零概率、K=0的Wald区间，再用21种K精确求和得到覆盖率。

```
print(.98**20)
print(e.wald_interval(0, 20))
print(e.exact_wald_coverage(.02, 20))
```

通过条件：全零概率约0.667608、区间[0,0]、覆盖率约0.3317923493。列出哪些k覆盖p，答案为1、2、3。这个结果是二项概率求和，不给它附加“模拟标准误”。在p网格上画曲线时，图上相邻点不平滑可能来自离散区间覆盖事件变化，而非数值算法出错。

解释把区间裁剪到[0,1]为何不会改变对合法p的覆盖事件。再说明“所有方法只要标95%就有95%覆盖”为什么已经被反例否定。

7. 先配对，再按个体重采样

读取data/paired.csv中的四对教学值。before=(1,2,4,8)，after=(2,3,5,9)。独立单位是subject，两列不能各自随机打乱。

```
before = [1,2,4,8]
after = [2,3,5,9]
paired = e.paired_bootstrap(before, after, 1999, 25025, 'paired')
independent = e.paired_bootstrap(before, after, 1999, 25025, 'independent')
print(e.sample_sd(paired), e.sample_sd(independent))
```

通过条件：paired每个值都严格等于1，标准差严格为0。independent分布有变化。用协方差公式解释这个区别；不可写成“配对法证明真实效果完全一样”。这组数据是为展示索引语义设计的极端合成样本。

进阶核验：before经验分布方差是7.1875，after相同。独立抽取两个各含4项的样本均值之差，其条件方差为 $2 \times 7.1875/4 = 3.59375$ 。用足够多复制近似这一数值，并注明复制有限，不能要求样本方差与理论逐位相等。

8. 按组与按行的覆盖对照

先手算三组数据：A=(0,0)、B=(2,2)、C=(4,4)。总体目标暂不讨论，先固定这份观察，计算两种Bootstrap的条件方差。按组等价于从(0,2,4)抽3次；按行等价于从(0,0,2,2,4,4)抽6次。两者单次经验方差都为8/3，所以均值复制方差分别为8/9和4/9。

随后运行主模拟。模型是 $X_{gj} = 3 + A_g + \varepsilon_{gj}$ ，G=24、m=6，A标准差2，独立噪声标准差0.5。每次外层先生成新的24组数据，内层各取B=399，外层重复1000次。真实目标均值为3。

通过条件：默认PCG64和种子下，行Bootstrap命中590次，组Bootstrap命中936次。写出各自MC标准误差约0.015553与0.007740，平均宽度约0.658888与1.566620。真均值方差为0.16840278，错误iid公式只有0.02951389。

默认确切命中数用于同环境复现核对，不是统计规律。不应靠修改种子寻找更接近0.95的一次结果。若软件版本或算法改变，应说明差异并重算，而不是强行覆盖输出。

9. 改参数前先写预测

把data/model_spec.json复制到自选文件，再用--spec传入。每次只改一个因素，并先写方向预测。

- 增大G但保持m：独立组数增加，两部分均值方差都下降。
- 增大m但保持G：独立噪声项下降，组效应项 τ^2/G 不变。
- 把 τ 设为0：组内共享效应消失，按行的独立假设重新合理；有限样本两个Bootstrap结果仍不必完全相等。
- 增大B：端点的Monte Carlo近似更稳定，不会修复按行抽取造成的结构错误。
- 增大R：测得覆盖率的MC误差大致按 $R^{-1/2}$ 缩小，不直接改变区间算法的真实覆盖率。

接口设置总运算预算，过大的组合会明确报错。不要删除预算后盲目扩大循环。比较不同方法的覆盖差时记录每次命中差；这两种方法在同一份外层样本上计算，不可当成两组独立比例。

10. 失败输入也要留下证据

下面每个调用都应该抛出ValueError，且不修改原始输入：

```

bad_calls = [
    lambda: e.sample_sd([1, 1, True]),
    lambda: e.bootstrap_mean([0, 2, float('nan')], B=99),
    lambda: e.bootstrap_mean([0,2], sigma='2'),
    lambda: e.weighted_quantile([0, float('nan')], [1,0], .5),
    lambda: e.paired_bootstrap([1,2], [2,True]),
    lambda: e.group_bootstrap([[0,0],[2,2,float('inf')]]),
]
for call in bad_calls:
    try:
        call()
    except ValueError:
        print('rejected')
    else:
        raise RuntimeError('invalid input accepted')

```

不要把bool转换成整数后再做这项测试，那已经丢掉原始类型。配置JSON还须在原始数值token阶段拒绝1e-400这类非零下溢，并拒绝重复键，避免后一个mu遮住前一个非法值。真正的0和0e-400仍可表示零。测试零权重记录、数组最后一项和未选分支，能发现“只检查被使用数据”的缺陷。另用含最后一行NaN的临时CSV运行脚本到一个带sentinel文件的临时输出目录，确认运行失败后sentinel字节不变、没有新report文件。

11. 提交清单

提交自己的短报告、运行后的Notebook和必要的参数文件。报告必须包含：已知 σ 的完整覆盖证明；Bootstrap完全枚举；Beta可信区间条件解释；Wald精确失败概率；配对与组的抽样单位；覆盖率、MC标准误、R、B、分位数约定和随机种子；一个你主动触发并解释的输入错误。

最重要的验收句是：“我知道这个区间模拟了什么重复机制，也知道它没有保证什么。”如果只会把函数名换成bootstrap，而不确定应抽人、抽组还是抽行，先停下来重新画采样结构。